Using ATM2/3 Sensor Hub White Paper

Revision History

Date	Version	Description
July 8, 2019	0.1	Initial version created.
March 13, 2020	0.2	Updated Figure 7, 8, 10, 11, 12, Table 2.
November 20, 2020	0.3	Updated Port 0, Port 1, and Data Comparison sections.
December 2, 2020	0.31	Corrected typos.
April 16, 2021	0.32	Updated format, corrected typos.

CONFIDENTIAL

©2019-2021, Atmosic Technologies Inc. All rights reserved. Atmosic logo is a registered trademark of Atmosic Technologies Inc.
All other trademarks are the properties of their respective holders. This white paper is subject to change without notice.

Table of Contents

Introduction 4

Sensor Hub Configuration 4
- Port 0 5
- Port 1 7

Data Comparison 8

Flash Control 10

Event Timing 11

Power Comparison 13
- Measurement methods 13
- Measurement data 13
  - Use Case 1: AT_APB_SHUB Polling 14
  - Use Case 2: ATM3 Active Polling 16
  - Data Extrapolation 17

Conclusion 18

Appendix 19

List of Figures

Figure 1 - Sensor Hub Port 0 Configuration
Figure 2 - Sensor Hub Port 1 Configuration
Figure 3 - Sensor Hub Port Trigger Enable
Figure 4 - Data Transfer of Sensors Connected to Port 0 and Port 1
Figure 5 - Data Read/Write Cycle Timing
Figure 6 - Flash Current Profile In Various Modes
Figure 7 - ATM3 VDD1D Current Profile
Figure 8 - ATM3 VDD1A Current Profile
Figure 9 - External Flash Current Profile In Sampling, Wait and Transmission Phases
Figure 10 - ATM3 VDD1D Current Profile In Sampling, Wait and Transmission Phases
Figure 11 - ATM3 VDD1A Current Profile In Sampling, Wait and Transmission Phases
Figure 12 - Average Power Over Time

CONFIDENTIAL

List of Tables

Table 1 - Sequence of Packets
Table 2 - Average Power Comparison Over Wait Time

CONFIDENTIAL

Introduction

In applications such as monitoring temperature, humidity, fluid velocity, etc, the IOT device reads data from one or more sensors, possibly aggregates it, and then transmits it over Bluetooth LE to an external host. The ATM3 series offers two methods to achieve this functionality and, depending on the specifics of the use case, one method may be more power efficient than the other. To maximize battery life, it is important that the right method be picked.

One method is to use the integrated CPU to manage everything. On a periodic basis the CPU wakes from a low power state, queries the sensors, and stores the resultant data in a system memory that will be retained throughout the next low power state. On a less frequent periodic basis, the CPU will also transmit the aggregated data off chip over Bluetooth LE. This method requires some device state to be retained and prevents the ATM3 from going to the lowest possible power state.

The second method is to use the integrated sensor hub. The sensor hub is a small micro-controller that can, through appropriate register configuration, read data from the sensors and store the resultant data into an external flash device. It does not need any assistance from the integrated CPU nor does it need any state retained. While the sensor hub has control of the ATM3 the vast majority of the ATM3 can be in the hibernate low power state. On a less frequent periodic basis, both the CPU and Bluetooth LE subsystems will wake from hibernate to transmit the aggregated data off chip over Bluetooth LE. As an even more power efficient optimization, the CPU and Bluetooth LE subsystems only need to wake when the sensor hub detects an alarm condition. An alarm condition is when a sensor reading exceeds a programmed threshold.

Fundamentally the sensor hub method relative to the CPU only method achieves lower power during idle modes at the expense of having to do complete boots when Bluetooth LE functionality is needed and at the expense of having to write to an external flash. The sensor hub becomes the more power efficient solution as the frequency of sensor reads decreases.

This white paper first describes how to program the sensor hub. It then does an average power comparison between the two methods using a realistic set of parameters. Actual customer use cases may differ but this white paper outlines a methodology that should be useful in all use cases.

Sensor Hub Configuration

At the core level, the connectivity to sensors is conceptualized as two ports. Each port can be configured to function in either the SPI or I2C protocol. At the system level, the user connects the sensors to the appropriate pins reserved for the sensor hub and programs the pin mux accordingly.

CONFIDENTIAL

Port 0

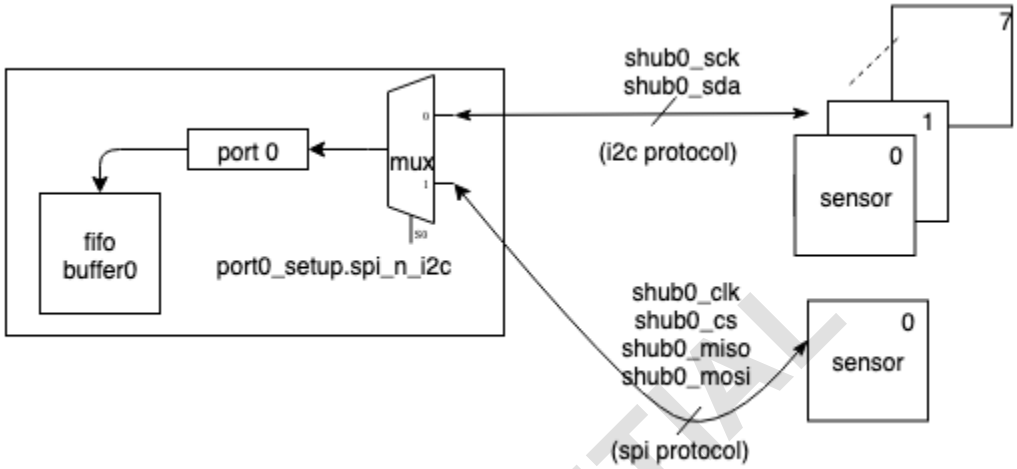


Figure 1 - Sensor Hub Port 0 Configuration

The first configuration for port 0 is in **port0_setup**[1] which specifies: the type of protocol (*spi_n_i2c*), the number of sensors connected to this port (*num_sensors*[2]), off/on state of the port (*port_en*) and whether the sensor data are to be compared with pre-programmed threshold quantities (*trig_en*).

If the protocol is SPI (*spi_n_i2c* = 1), only one sensor is expected and the user must program the sensor opcode (**port0_spi_eng0_ctrl**) and the number of bytes (**port0_sensor0_setup**) to be read from the sensor. Note that an extra byte will be read from the sensor and written to flash, so it should be discarded by the application.

If the protocol is I2C *(spi_n_i2c* = 0), up to 8 sensors can be connected to port 0 as specified in **port0_setup**.*num_sensors*. For each sensor, the corresponding **port0_sensorx_setup** register should be programmed with the number of bytes to be read from the sensor minus one. Furthermore, the user must program the control registers for I2C packets, each of which corresponds to one of 4 stages making up for one sequence of I2C data transfer from a sensor. These 4 stages are <sensor id>, <pointer address in the sensor>, <sensor_id>, <sensor data>, and the last stage (sensor data) is replicated as many times as the number of bytes of data to be read from a sensor device. These stages are programmed in **port0_i2c_idw_x, port0_i2c_addr_x, port0_i2c_idr_x, port0_i2c_dat_ctrl, port0_i2c_dat_last_ctrl**.

[1] Text in bold is register name
[2] Text in italics is register field name

CONFIDENTIAL

Note that the data stage is controlled by two registers: one for the first and intermediate bytes and the other for the last byte in a read sequence. If there is only one byte of data, then it is considered as the last byte and **port0_i2c_dat_last_ctrl** is used. The x sign denotes the ordinal numbers respective to the set of sensors connected to port 0. Note that the two registers in the data stage are common for all sensors.

Table 1 - Sequence of Packets

Packet 1	Packet 2	Packet 3	Packet 4	Packet 5
ID(W)*	ADDR*	ID(R)*	DATA	DATA_LAST

[3]

a. Packet 1 is a request to write and contains the sensor device ID. This packet is constructed by **port0_i2c_idw_x**, where x corresponds to the sensor number. In most cases, the fields in this register are programmed as shown below:

i. *ext_data_o* = 7 bits device ID, 1 bit to denote Write (0 for most cases)
ii. *ext_data_oe* = 0xff
iii. *ext_head* = 0
iv. *ext_tail* = 1
v. All other fields = 0

b. Packet 2 is a write packet containing the starting address location of the register inside the sensor device. This packet is constructed by **port0_i2c_addr_x**, where x denotes a specific sensor. In most cases, the fields in this register are programmed as shown below:

i. *ext_data_o* = 8 bits of the address value
ii. *ext_data_oe* = 0xff
iii. *ext_head* = 1
iv. *ext_tail* = 2
v. All other fields = 0

c. Packet 3 is a request to read and contains the sensor device ID. This packet is constructed by **port0_i2c_idr_x**, where x corresponds to the sensor number. In most cases, the fields in this register are programmed as shown below:

i. *ext_data_o* = 7 bits device ID, 1 bit to denote Read (1 for most cases)
ii. *ext_data_oe* = 0xff
iii. *ext_head* = 0
iv. *ext_tail* = 1
v. All other fields = 0

d. Packet 4 is a data packet. This packet is constructed by **port0_i2c_dat_ctrl**. This packet is replicated multiple times to acquire multiple bytes, except for the last byte. In most cases, the fields in this register are programmed as shown below:

i. *ext_data_o* = 0
ii. *ext_data_oe* = 0

[3] * Denotes 0 through 7, the eight I2C sensors supported by port 0, one I2C sensor is supported by port 1.

CONFIDENTIAL

iii. *ext_head* = 1
iv. *ext_tail* = 1
v. All other fields = 0

e. Packet 5 is a data packet. This packet is constructed by **port0_i2c_dat_last_ctrl** to acquire the last byte from the sensor. In most cases, the fields in this register are programmed as shown below:

i. *ext_data_o* = 0
ii. *ext_data_oe* = 0
iii. *ext_head* = 1
iv. *ext_tail* = 0
v. All other fields = 0

Port 1

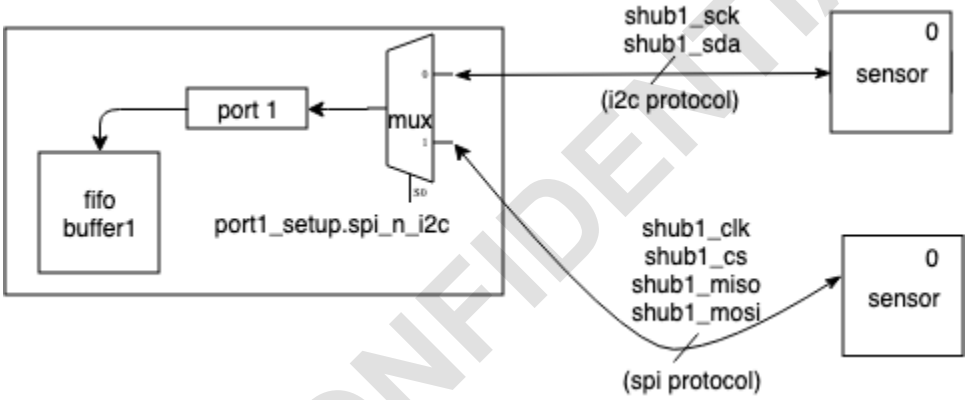


Figure 2 - Sensor Hub Port 1 Configuration

The first configuration for port 1 is in **port1_setup** which specifies: the type of protocol (*spi_n_i2c*), the number of sensors connected to this port (*num_sensors*), off/on state of the port (*port_en*) and whether the sensor data are to be compared with pre-programmed threshold quantities (*trig_en*).

If the protocol is SPI (*spi_n_i2c* = 1), only one sensor is expected and the user must program the sensor opcode (**port0_spi_eng0_ctrl**) and the number of bytes (**port0_sensor0_setup**) to be read from the sensor. Note that an extra byte will be read from the sensor and written to flash, so it should be discarded by the application.

If the protocol is I2C (*spi_n_i2c* = 0), only one sensor can be connected to port 1 and the **port1_sensor0_setup** register should be programmed with the number of bytes to be read from the sensor minus one. Furthermore, the user must program the control registers for I2C packets, each of which corresponds to one of 4 stages making up for one sequence of I2C data transfer from a sensor. These 4 stages are <sensor id>, <pointer address within the sensor>,

CONFIDENTIAL

<sensor_id> <sensor data> ... <sensor data>, whereby the last stage (sensor data) can be multiple packets equal to the number of bytes programmed. These stages are programmed in **port1_i2c_idw_0, port1_i2c_addr_0, port1_i2c_idr_0, port1_i2c_dat_ctrl, port1_i2c_dat_last_ctrl**. The data stage is specified in two registers: one for the first and intermediate bytes and the other for the last byte in each read sequence.
Programming of the registers are the same as explained in port 0 section, except that port 1 only supports one sensor in either protocol.

Data Comparison

Port 0 is equipped with one alarm for sensor 0 when only one sensor is connected. Port 1 is equipped with one alarm for the single sensor that can be attached to it. The alarm on port 0 is explained in this section. The alarm treats the stream of bytes from sensor 0 as a sequence of distinct quantities, each of which is made up of a number of contiguous bytes.

The user can define up to 4 quantities which are determined by their sizes (**port0_alm0_quan_ctrl**.*quanx_size*) and starting positions (**port0_alm0_quan_ctrl**.*quanx_pos*) in the byte stream (up to 32 bytes). These are the constraints on the quantities' definitions:

quan3_pos > *quan2_pos* + *quan2_size*
quan2_pos > *quan1_pos* + *quan1_size*
quan1_pos > *quan0_pos* + *quan0_size*

1<*quanx_size* < 3 : each quantity can be at most 3 bytes, so the combined number of bytes is at most 12 bytes.

If the quantities have their most significant byte sent in first, the user must be programmed (**port0_alm0_quan_ctrl**.*lsb_first*) to 0. Also, **port0_alm0_quan_ctrl**.*quan_dat_is_signed* specifies whether the quantities are signed numbers.

Next is to set the floor and ceiling reference quantities with which the sensor's quantities are compared. Twelve bytes are allocated for the floor and the ceiling. For the ceiling reference, the twelve bytes, *byte_max_00.... byte_max_11*, are spread across registers **port0_alm0_thrhld_max_0**, **port0_alm0_thrhld_max_1** and **port0_alm0_thrhld_max_2**. For the floor reference, the twelve bytes, *byte_min_00.... byte_min_11*, are spread across registers **port0_alm0_thrhld_min_0**, **port0_alm0_thrhld_min_1** and **port0_alm0_thrhld_min_2**.

For each reference, the hardware packs the bytes together into quantities in similar fashion as it does to the sensor's quantities, except that the packing is the same irrespective of the setting of lsb_first. Each reference quantity is determined by the *quanx_size*.

CONFIDENTIAL

For instance, if *quan0_size* = 1, *quan1_size* = 2 , *quan2_size* = 2 and *quan3_size* = 0, then the grouping are as follows:

a. For the minimum references:
   {*byte_min_00*, *byte_min_01*},
   {*byte_min_02*, *byte_min_03*, *byte_min_04*},
   {*byte_min_05*, *byte_min_06*, *byte_min_07*},
   {*byte_min_08*}

b. For the maximum references:
   {*byte_max_00*, *byte_max_01*},
   {*byte_max_02*, *byte_max_03*, *byte_max_04*},
   {*byte_max_05*, *byte_max_06*, *byte_max_07*},
   {*byte_max_08*}

All 4 sensor quantities are then compared to their references and have their out of range status captured in **port0_status**.*out_of_range_stat[3:0]*. For each bit, once set, it remains set until the next sensor hub activation.

The total number of possible values of the 4-bit *out_of_range_stat* is 15 (disregarding 0). The user can choose any value to wake up the system from hibernation by programming the desired value in **port0_alm0_trig_ctrl**.

Each port has its trigger enabled in **portx_setup**.*trig_en* as depicted in the diagram below. The combined trigger is used to wake up the ATM3 from hibernation.

CONFIDENTIAL

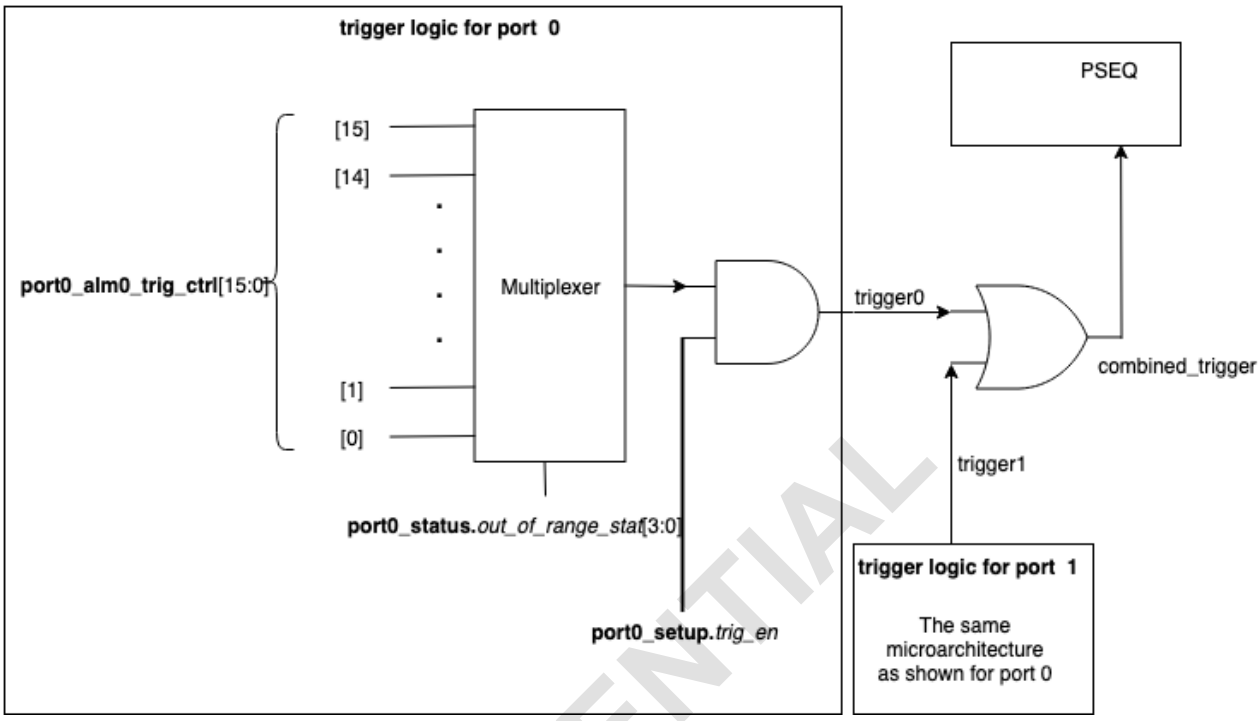


Figure 3 - Sensor Hub Port Trigger Enable

Flash Control

The sensor data is written to the external flash chip, which can be Macronix, Winbond or Micron.

The user must specify the following items for the flash operation:

a. Opcodes in **flash_ctrl0**: for write enable, page program, power down and release from power down. For Macronix and Winbond, **flash_ctrl3**.*is_macronix* must be 1. Further, for Winbond, **flash_ctrl3**.*serialize_pp_address* must be 1.

b. Start address in **flash_ctrl1**: this is the address in the flash where the first data byte is written to.

c. Max address in **flash_ctrl2**: this is the last address for sensor data. The sensor hub finite state machine (FSM) will wake up the CPU when this address is reached. The number of leftover data bytes are stored in **lost_write_cnt** and **flash_status0**.*wr_saturated* is set by the hardware. These status registers can be cleared by writing ones to **flash_ctrl3**.*clear_lost_write_cnt* and **flash_ctrl3**.*clear_flash_overflow*, respectively.

d. Interface mode in **flash_ctrl3**.*qspi_mode*: whether the interface is single SPI or quad SPI.

CONFIDENTIAL

e. Page program delay time in **flash_ctrl4**.*pp_delay*: after a page program opcode, the flash takes some time to complete the write process. This page program latency is specified in the flash specification, so the user shall program this delay time accordingly.

f. Power cycling mode: this mode is controlled by setting **flash_ctrl3**.*power_cycle_en* to 1. In this mode, the flash chip is put into a low power state after each write and recalled to active state at the start of the sensor read phase. To account for the flash recovery time from the low power state, the user can program for a delay time in **flash_ctrl4**.*pd_delay*. The write phase only starts if both the read phase and delay **flash_ctrl4**.*pd_delay* are completed. For Winbond and Micron, **flash_ctrl3**.*rpd_has_clock* must be 1.

Event Timing

The sensor hub main FSM is triggered by the program sequencer (PSEQ). Once triggered, the FSM reads the data from the sensors and writes to the flash repeatedly until the PSEQ requests for a stop. The cycle duration is set by **setup1** which shall be programmed longer than the time it takes to read all sensors' data and write to flash.

In each cycle (**setup1**), sensors on port 0 and port 1 are read simultaneously. Starting with sensor 0, the FSM reads the data from both ports and writes port 0's data and then port 1's data to the flash. For subsequent sensors, the FSM only receives the data from port 0.

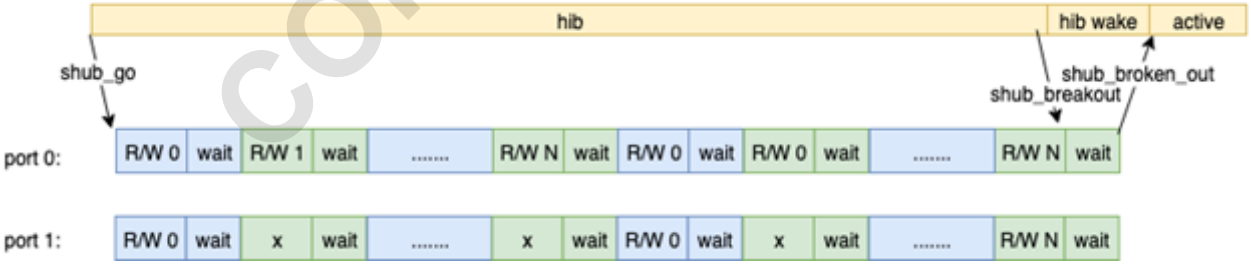


Figure 4 - Data Transfer of Sensors Connected to Port 0 and Port 1

In the illustration above, S sensors are connected to port 0 and are denoted by a range of numbers from 0 to S, where S <=7, and 1 sensor is connected to port 1. The sensor hub reads the data from sensors #0 on both ports simultaneously and write the data from port 0 and port 1, in that order, into the external flash. Subsequent sensors on port 0 are then polled for their data while the sole sensor on port 1 is not. Then the sensor hub wraps around to sensors #0. The sensor hub maintains data collection until the PSEQ core interrupts it with **shub_breakout**. The sensor hub will finish the remaining write to flash and terminate.

CONFIDENTIAL

The detailed timing of a data read/write cycle is described below:

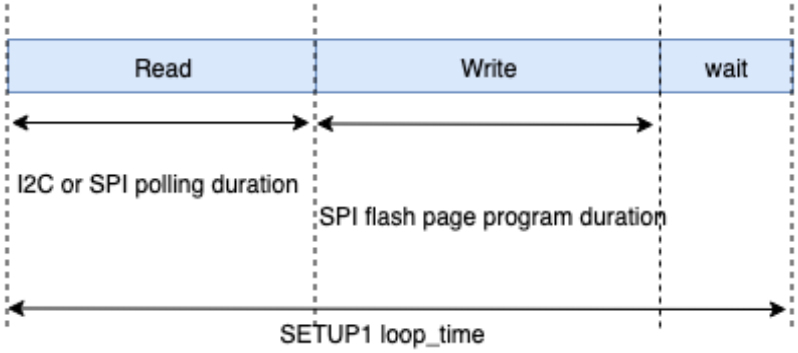


Figure 5 - Data Read/Write Cycle Timing

The register **setup1** controls the outer loop duration of a read write pair and its associated wait time. Inside the loop, the read duration in terms of clock cycles is:

Read SPI (cycles) = 65 * (B + 3) + 4

Read I2C (cycles) = 8 * (B+1) + 5

where B is the number of bytes polled from a sensor device. Because there are two data streams from two ports, the duration is determined by the longest of the two.

Write duration is the sum of the duration of each of the following components:
write enable opcode (wren), page program opcode (pp), the address (addr), the data (data), page program delay (PP_DELAY), power down opcode (pd) and power down delay (PD_DELAY).

1. wren (cycles) = 8/n + 4
2. pp (cycles) = 8/n + 2
3. addr (cycles) = 24/k
4. data (cycles) = N*8/m
5. PP_DELAY = **flash_ctrl4**.*pp_delay* +3
6. pd (cycles) = 8/n + 3
7. PD_DELAY = **flash_ctrl4**.*pd_delay* +1

The variables in the equations above are determined as follows:

N: number of bytes per round polled from both ports.
n: 1 for Macronix, Winbond, and Single-SPI Micron, 4 for Quad-SPI Micron
k: 1 for Winbond, Single-SPI Macronix, and Single-SPI Micron, 4 for Quad-SPI Macronix and Micron
m:1 for Single-SPI, 4 for Quad-SPI
PP_DELAY: programmable delay to account for the flash's page program latency.

There is one pitfall in the page program transaction. The N bytes of data (the total number of bytes from port 0 and port1 in one polling round) is written to the flash in two transactions if they happen to straddle the boundary of two flash pages (256 bytes per page). The first transaction writes until the flash address reaches the end of a boundary, denoted by the last 8 bits equal to

CONFIDENTIAL

0xff. The second transaction finishes the remaining data bytes in a new page. This constraint is imposed by flash device specifications from Macronix, Micron and Winbond. If the loop-time is long enough, then this pitfall will not show any effect on the periodicity of the polling cycles.

Power Comparison

Measurement methods

An experiment was conducted on two cases for a power comparison analysis. In both cases, the activity of ATM3 can be summarised as follows:

1. Poll 16 bytes of data and write it to a storage space.
2. Wait for a programmed duration.
3. Repeat step 1 until 4 KB of data have been collected.
4. Transmit the accumulated data over Bluetooth

The external sensor is assumed to be on all the time and thus would be consuming the same amount of energy.

In use case 1, the user configures the sensor hub core, at_apb_shub, to collect the entire 4 KB while the rest of ATM3 is in the low power state of hibernation. The pseudo code to program the sensor hub core is shown in the appendix.

In use case 2, ATM3 polls the sensor data using one of the two I2C cores and then waits in the low power state of retention in which state elements are retained in a low power state. Note that hibernation is not a viable low power state for this method because SRAM blocks must be retained during the wait time. The poll and wait cycle is repeated until the entire 4 KB of data is completed.

CONFIDENTIAL

Measurement data

For both exercises, currents on 3 power supplies are measured:

a. Flash current consumed by the external flash device MX25R4035F at 3.2V
b. VDD1D current consumed by the digital logic in ATM3.
c. VDD1A current consumed by the analog circuit in ATM3.

Use Case 1: AT_APB_SHUB Polling

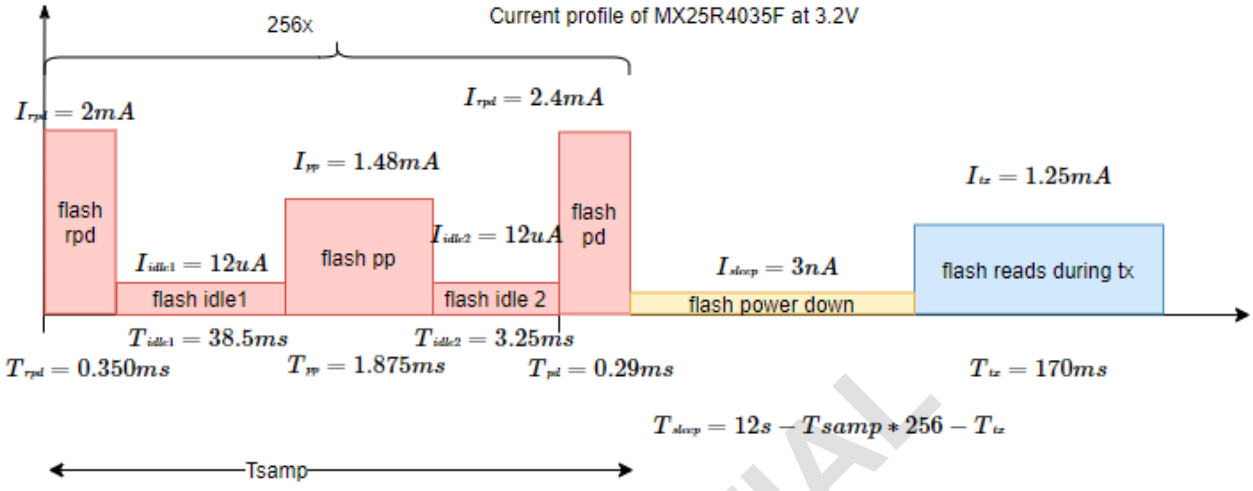


Figure 6 - Flash Current Profile In Various Modes

The above diagram illustrates the current profile consumed by the external flash in various modes of flash operation:

a. flash rpd: the flash device is released from power down mode to be ready for page program.
b. flash idle1: the sensor data is being polled during this time.
c. flash pp: flash page program of 16 bytes of sensor data to the flash.
d. flash idle2: the delay duration after page program as specified by Macronix.
e. flash pd: the flash device receives an instruction to go into power down mode.
f. flash power down: the period of absolute low power.
g. flash read during tx: flash current activity during Bluetooth transmission.

As shown the profile is divided into 3 distinct phases:

a. Sampling phase: marks the activity of sensor data polling and storing period. This activity is followed by the power down phase for 256 times. But for computation simplification, the sampling and the power down phases are lumped into two separate groups. The duration of the power down phase (Tsleep) is arbitrarily chosen to make the entire time of polling and transmitting 4 KB of data to be 12 seconds.
b. flash power down: marks the wait duration after the sampling.
c. flash read during tx: shows the final phase where the 4 KB data are transmitted

CONFIDENTIAL

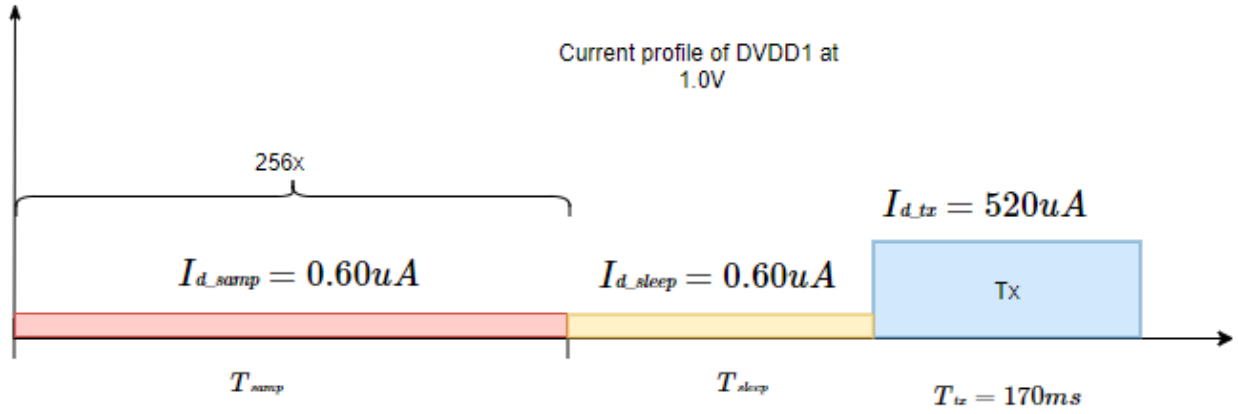


Figure 7 - ATM3 VDD1D Current Profile

The above diagram illustrates the current profile of VDD1D which is the main supply to the digital logic in ATM3. ATM3 is put into hibernation mode while the sensor hub core polls and stores the sensor data. Therefore, the current consumption remains low and constant during Tsamp and Tsleep. Only when the transmission time that ATM3 becomes fully active.

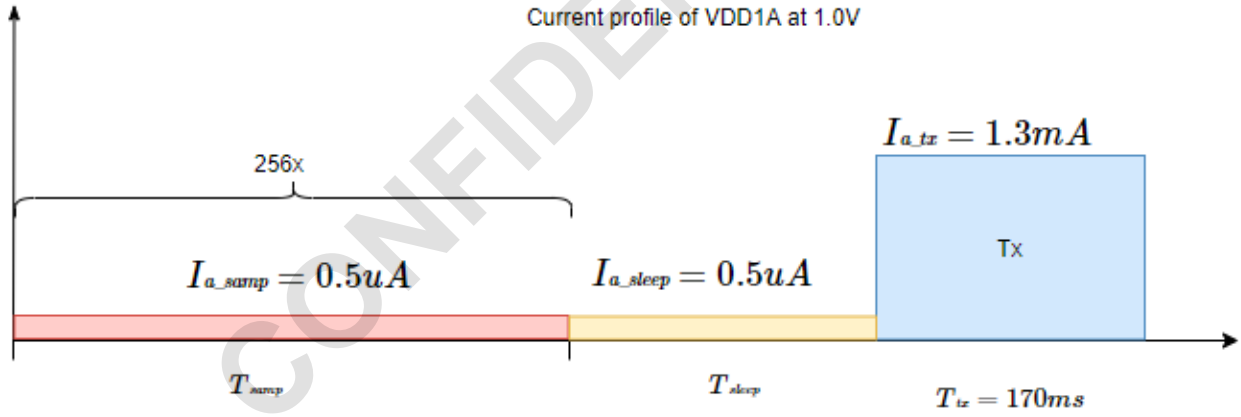


Figure 8 - ATM3 VDD1A Current Profile

The above diagram illustrates the current profile of VDD1A which is the supply to the analog circuit.

CONFIDENTIAL

Use Case 2: ATM3 Active Polling

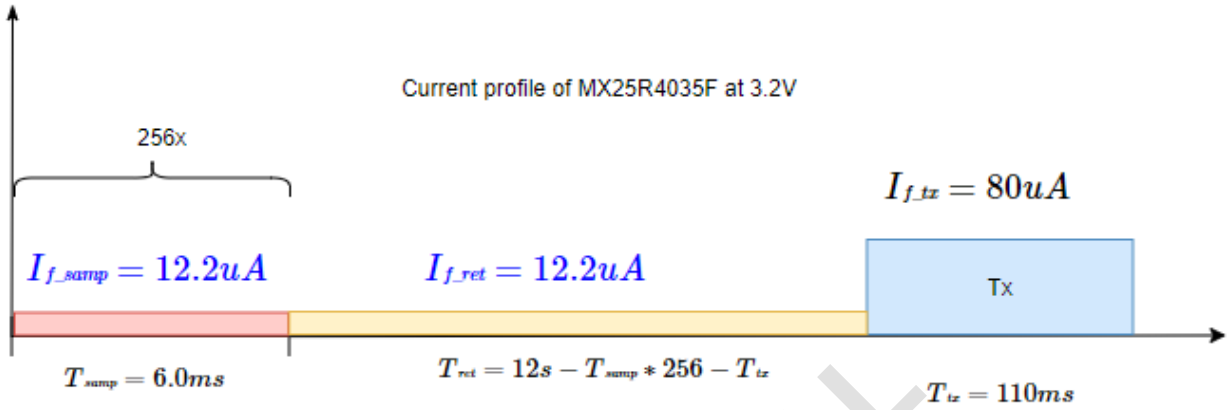


Figure 9 - External Flash Current Profile In Sampling, Wait and Transmission Phases

The diagram above shows the current profile consumed by the external flash in three distinct phases: the sampling phase, the wait phase (ATM3 is put into retention) and the transmission phase. In practice, the sampling and wait phases are repeated in tandem, but for ease of computation, both are separated into two separate regions. The duration of retention is arbitrarily made so that the entire time duration is 12 seconds.

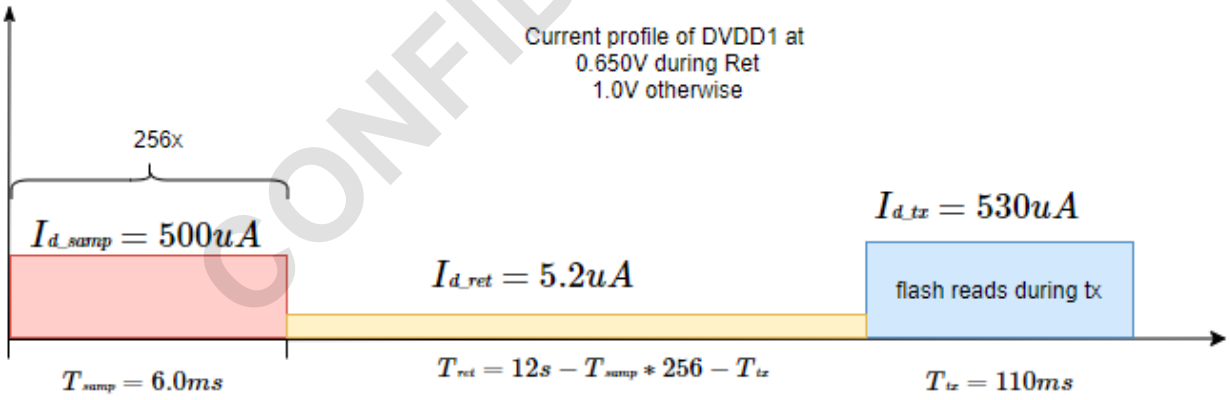


Figure 10 - ATM3 VDD1D Current Profile In Sampling, Wait and Transmission Phases

The above diagram shows the current profile of VDD1D in 3 phases as described in the flash current profile section.

CONFIDENTIAL

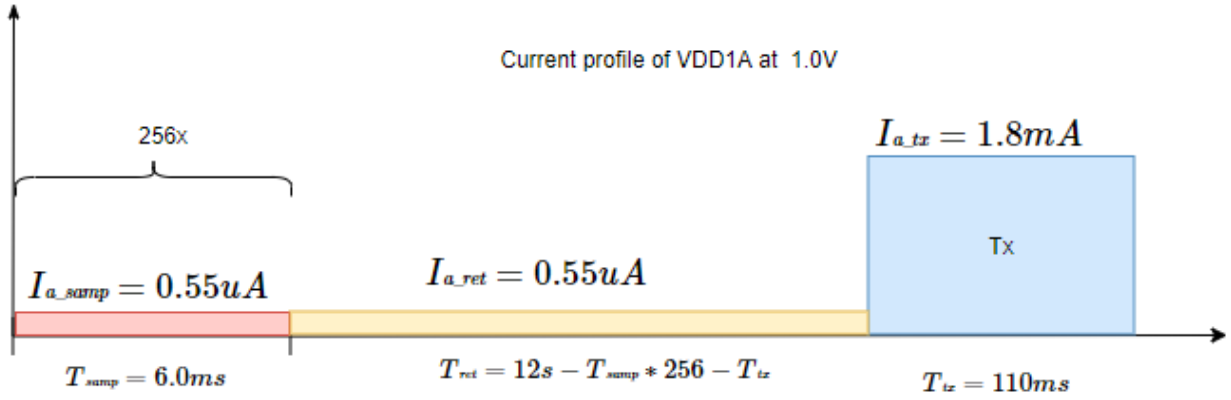


Figure 11 - ATM3 VDD1A Current Profile In Sampling, Wait and Transmission Phases

Last is the current profile of VDD1A in the 3 phases.

Data Extrapolation

Given the current profiles shown in the previous section, the average power can be computed in each method as a function of additional wait time. The wait time is denoted as Tsleep in the first method and Tret in the second method.

For each method, the total energy consumed by all three supplies over the 12 seconds period was computed and called Efixed. The average power over time is then:

$$P(t) = \frac{E_{fixed} + I_{d_sleep}V_{d}t + I_{a_sleep}V_{a}t + I_{flash_sleep}V_{flash}t}{T_{fixed} + t}$$

Note that t denotes the total additional wait time that is spread out between each sampling of sensor data (16 bytes).

The following graph shows the two average power over time.

CONFIDENTIAL

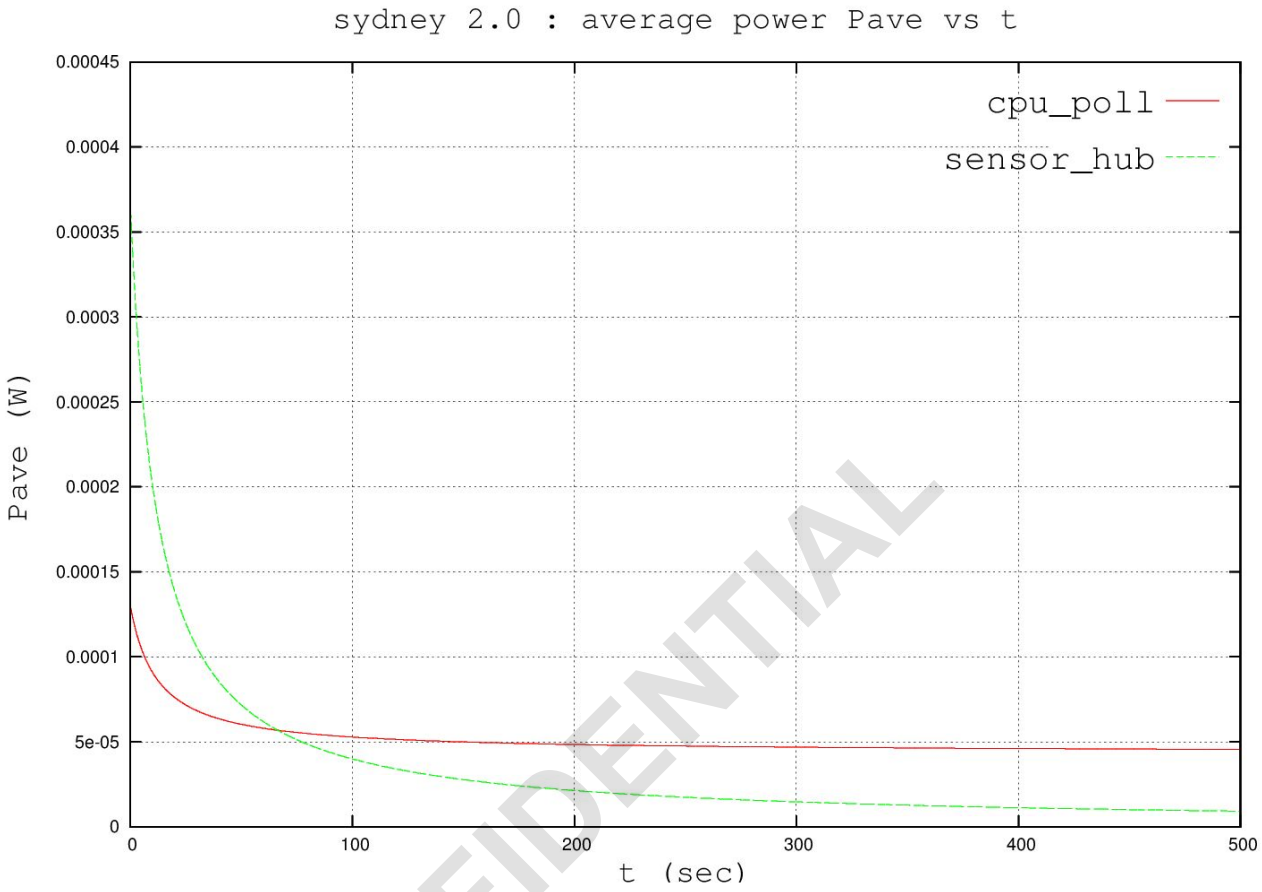


Figure 12 - Average Power Over Time

Table 2 - Average Power Comparison Over Wait Time

Additional total wait time used in polling 4 KB	0s	30s	60s	90s	150s	600s	infinity
Power ratio: sensor_hub / cpu_poll	282%	150%	106%	81%	55%	17%	1.1%

Conclusion

As shown in the graphs, using the sensor hub becomes more attractive from the energy standpoint as the additional wait time exceeds 70 seconds. This experiment serves as a rough guideline on which method to use depending on the total time needed to poll for data in the range of KB.

CONFIDENTIAL

Appendix

This appendix provides the pseudo code for the configuration of use case 1 where sensor hub core collects the sensor data and stores them into an external flash device during hibernation.

```
// This program has two distinct parts:
// 1. transmits the sensor data from a flash sector via Bluetooth LE.
// 2. collects sensor data and store them into a flash sector during hibernation.
// Upon waking up from hibernation, the program restarts at the beginning of main.

int main(void) {
    //transmission of sensor data:
    //code is not shown here.


    //=============================================================================================
    // Below pseudocode programs the sensor hub
    // turn pseq out of reset
    CMSDK_WRPR1->APB8_CTRL = CLK_ON_RESET_ON;
    CMSDK_WRPR1->APB8_CTRL = CLK_ON_RESET_OFF;

    // clock gate off and power down cores not under hardware control
    CMSDK_PSEQ->KSMQDEC_CONTROL = 0; // turns off clock
    CMSDK_PSEQ->WURX_CONFIG=0; // powers down wake-up receiver
    CMSDK_PSEQ->GADC_CONFIG=0; // powers down GADC

    // erase 4kB flash sector needed for sensor hub data storage
    // code not shown here

    // enable sensor hub functionality
    CMSDK_PSEQ->SENSOR_HUB_CONTROL =
         PSEQ_SENSOR_HUB_CONTROL__SHUB_CLKEN__WRITE(1) |
         PSEQ_SENSOR_HUB_CONTROL__ENABLE_FUNC__WRITE(1);


    // pin mux configuration to configure a set of pins to function sensor hub's i2c0
    pin_mux_config();

    //-------------------------------------------------------------------------
    // the following steps configure the sensor hub
    //-------------------------------------------------------------------------
    CMSDK_SHUB->SETUP1      = SHUB_SETUP1__LOOP_TIME__WRITE(loop_time); // e.g loop_time = 4000

    // port 0 configuration
    CMSDK_SHUB->PORT0_SETUP =
         SHUB_PORT0_SETUP__NUM_SENSORS__WRITE(0) | // number of sensors = 1
         SHUB_PORT0_SETUP__SPI_N_I2C__WRITE(0) |   // select i2c protocol
         SHUB_PORT0_SETUP__PORT_EN__WRITE(1);      // enable port 0
    CMSDK_SHUB->PORT0_SENSOR0_SETUP =
         SHUB_PORT0_SENSOR0_SETUP__NUM_BYTES__WRITE(15); //number of bytes in sensor 0

    // configure i2c protocol for sensor0 on port_0 (elaborated in the white paper main text above)
    dev_addr = DEVICE_ID;
    pntr = ptr; // starting address of data bytes in the sensor register space
    dev_addr_W = (dev_addr << 1) + 0; // 0 denotes Write
    dev_addr_R = (dev_addr << 1) + 1; // 1 denotes Read

    odat0 = (0xff << 8) + dev_addr_W;
    odat1 = (0xff << 8) + pntr;
    odat2 = (0xff << 8) + dev_addr_R;
    odat3 = 0x0;  // master is going to read back from slave
    odat4 = 0x0;  // master is going to read back from slave
```

CONFIDENTIAL

```
    head0 = 0; tail0 = 1;
    head1 = 1; tail1 = 2;
    head2 = 0; tail2 = 1;
    head3 = 1; tail3 = 1;
    head4 = 1; tail4 = 0;
    CMSDK_SHUB->PORT0_I2C_IDW_0         = odat0<<1 | tail0<<21 | head0<<19;
    CMSDK_SHUB->PORT0_I2C_ADDR_0        = odat1<<1 | tail1<<21 | head1<<19;
    CMSDK_SHUB->PORT0_I2C_IDR_0         = odat2<<1 | tail2<<21 | head2<<19;
    CMSDK_SHUB->PORT0_I2C_DAT_CTRL      = odat3<<1 | tail3<<21 | head3<<19 | 1<<23;
    CMSDK_SHUB->PORT0_I2C_DAT_LAST_CTRL = odat4<<1 | tail4<<21 | head4<<19 | 1<<23 | 1<<24;

    // flash interface configuration
    CMSDK_SHUB->FLASH_CTRL0 =
         SHUB_FLASH_CTRL0__OPCODE_RPD__WRITE(0xab)|
         SHUB_FLASH_CTRL0__OPCODE_PD__WRITE(0xb9) |
         SHUB_FLASH_CTRL0__OPCODE_PP__WRITE(0x38) |
         SHUB_FLASH_CTRL0__OPCODE_WREN__WRITE(0x06);
    CMSDK_SHUB->FLASH_CTRL1 =
         SHUB_FLASH_CTRL1__WR_START_ADDRESS__WRITE(wr_start_address);// the start address of a 4K sector in the flash
device
    CMSDK_SHUB->FLASH_CTRL2 =
         SHUB_FLASH_CTRL2__WR_MAX_ADDRESS__WRITE(wr_max_address); // the end address of the 4K sector
    CMSDK_SHUB->FLASH_CTRL3 =
         SHUB_FLASH_CTRL3__RPD_HAS_CLOCK__WRITE(1)|
         SHUB_FLASH_CTRL3__SERIALIZE_PP_ADDRESS__WRITE(0)|
         SHUB_FLASH_CTRL3__QSPI_MODE__WRITE(2)| // specify quad SPI
         SHUB_FLASH_CTRL3__IS_MACRONIX__WRITE(1)| // the external flash is Macronix
         SHUB_FLASH_CTRL3__POWER_CYCLE_EN__WRITE(1); // flash deep sleep cycling enabled
    CMSDK_SHUB->FLASH_CTRL4 =
         SHUB_FLASH_CTRL4__PD_DELAY__WRITE(0x2)|
         SHUB_FLASH_CTRL4__PP_DELAY__WRITE(104); // some delay after the page program

    //-------------------------------------------------------------------------
    // the following steps configure the power sequencer(pseq) for hibernation
    //-------------------------------------------------------------------------
    // set up the pseq's count down timer
    // it's expiration will be what wakes up ATM3 from hibernation state
    CMSDK_PSEQ->INIT_COUNT_DOWN = HIBERNATE_TIME; // calibrated so that the entire 4KB of data are polled by the sensor
hub
                                                  // In this use case:
                                                  // a. the sensor hub collects 16B per round in one loop_time of
duration
                                                  // b. total data of 4096B requires 256 rounds
                                                  // c. thus, HIBERNATION_TIME = loop_time*256

    // load expiration timer -
    // let's put a rising edge on load signal
    CMSDK_PSEQ->COUNTER_CONTROL = 1<<1;
    // don't advance until we know the 32kHz domain has seen load request and has done the load
    do {
         rdat = (CMSDK_PSEQ->INST_PENDING & 0x10000)>>16;
    } while (rdat == 1);
    CMSDK_PSEQ->COUNTER_CONTROL = 0<<1;

    // set wake mask; only cntdown will do the wave
    CMSDK_PSEQ->HIB_WAKE_MASK = (1<<1);  // timer expiring (and nothing else) will wake from hibernate

    // let's trigger the pseq to take the power state down
    CMSDK_PSEQ->CTRL0 =
        PSEQ_CTRL0__PINSEL_LATCH_OPEN__MASK |
        PSEQ_CTRL0__REQ_HIBERNATE_STATE__MASK |
        PSEQ_CTRL0__MANAGE_XTAL__MASK;

    //the power sequencer (PSEQ) is going to start pulling powers down;
```

CONFIDENTIAL

```
    //need to get into WFI before this happens
      __WFI();

    // When hibernation duration is completed, the program restarts at main
}
```

CONFIDENTIAL



CONFIDENTIAL

ATMOSIC TECHNOLOGIES - DISCLAIMER

This product document is intended to be a general informational aid and not a substitute for any literature or labeling accompanying your purchase of the Atmosic product. Atmosic reserves the right to amend its product literature at any time without notice and for any reason, including to improve product design or function. While Atmosic strives to make its document accurate and current, Atmosic makes no warranty or representation that the information contained in this document is completely accurate, and Atmosic hereby disclaims (i) any and all liability for any errors or inaccuracies contained in any document or in any other product literature and any damages or lost profits resulting therefrom; (ii) any and all liability and responsibility for any action you take or fail to take based on the information contained in this document; and (iii) any and all implied warranties which may attach to this white paper, including warranties of fitness for particular purpose, non-infringement and merchantability. Consequently, you assume all risk in your use of this white paper, the Atmosic product, and in any action you take or fail to take based upon the information in this datasheet. Any statements in this document in regard to the suitability of an Atmosic product for certain types of applications are based on Atmosic's general knowledge of typical requirements in generic applications and are not binding statements about the suitability of Atmosic products for any particular application. It is your responsibility as the customer to validate that a particular Atmosic product is suitable for use in a particular application. All content in this document is proprietary, copyrighted, and owned or licensed by Atmosic, and any unauthorized use of content or trademarks contained herein is strictly prohibited.

Copyright ©2019-2021 by Atmosic Technologies. All rights reserved.

www.atmosic.com